

Task 1 — The 6-DOF Quadcopter Model, Explained

HACKSIMUL8 2026 · PES University · 2 September 2026 Status: COMPLETE and NUMERICALLY VERIFIED

What Task 1 asked: "Develop the dynamic mathematical model of the 6 DOF UAV Quadcopter in MATLAB and SIMULINK in terms of state space equations."

Three deliverables are hidden in that sentence: a **dynamic model**, in **state-space** form, and **evidence that it is correct**. This document covers all three.

1. What "6-DOF" actually means

A rigid body free in space has **six degrees of freedom**:

	Degrees of freedom	Symbol
Translation	move along x, y, z	$\xi = [X, Y, Z]$
Rotation	rotate about x, y, z	$\eta = [\varphi, \theta, \psi]$

- **φ (roll)** — rotation about the body x-axis
- **θ (pitch)** — rotation about the body y-axis
- **ψ (yaw)** — rotation about the body z-axis

The central difficulty: the quadcopter has **6 degrees of freedom but only 4 actuators**. It is *underactuated*. It physically cannot slide sideways — it must **tilt** to move horizontally. That single fact shapes the entire control problem, and it appears in the model as the coupling $\ddot{x} = g \cdot \theta$.

2. Two coordinate frames

The problem statement's Fig. 1 shows both:

- **Inertial (ground) frame** $0\text{-}xyz$ — fixed to the earth. Position and altitude are measured here.
- **Body frame** $0_B\text{-}x_B\ y_B\ z_B$ — glued to the airframe, origin at the centre of gravity. **Thrust always acts along $+z_B$** , whatever attitude the vehicle is in.

Everything in the model comes from this one tension: *forces are generated in the body frame, but motion is measured in the inertial frame*. The rotation matrix links them.

The three assumptions stated in the problem are what let us do this:

1. homogeneous and symmetric block → the inertia tensor is **diagonal**, $I = \text{diag}(I_{xx}, I_{yy}, I_{zz})$
2. centre of the quadcopter is the centre of gravity → no offset terms
3. transmission absolutely rigid, elasticity neglected → no flexible-body dynamics

3. From rotor speeds to forces and torques

Each rotor spins at ω_i and produces:

- **thrust** $f_i = k \cdot \omega_i^2$ along $+z_B$ ($k = 2.996 \times 10^{-6} \text{ N}\cdot\text{s}^2$)
- **reaction torque** $\tau_i = b \cdot \omega_i^2$ about z_B ($b = 1.260 \times 10^{-7} \text{ N}\cdot\text{m}\cdot\text{s}^2$)

Both scale with ω^2 , which is what makes the quadcopter nonlinear at the input.

Rotor positions (from Fig. 1)

The figure shows a **cross / "plus" configuration** — the arms lie along the body axes:

Rotor	Position	Spin
1	(l, 0, 0) — on $+x_B$	one way
2	(0, -l, 0) — on $-y_B$	other way
3	(-l, 0, 0) — on $-x_B$	one way
4	(0, l, 0) — on $+y_B$	other way

Rotors 1 and 3 spin opposite to 2 and 4, so in hover their reaction torques **cancel** and the quadcopter does not spin about z .

Deriving the moments with $\tau = r \times F$

Take rotor 4 at $r_4 = (0, l, 0)$ with thrust $F_4 = (0, 0, f_4)$:

$$\tau_4 = r_4 \times F_4 = \begin{vmatrix} i & j & k \\ 0 & l & 0 \\ 0 & 0 & f_4 \end{vmatrix} = (l \cdot f_4, 0, 0)$$

So rotor 4 gives a **positive roll torque**. Rotor 2 at $(0, -l, 0)$ gives $(-l \cdot f_2, 0, 0)$. Doing this for all four rotors gives the **four virtual control inputs**:

$U1 = k(\omega_1^2 + \omega_2^2 + \omega_3^2 + \omega_4^2)$	total thrust	[N]
$U2 = k(\omega_4^2 - \omega_2^2)$	roll input	$\rightarrow \tau_\phi = l \cdot U2$
$U3 = k(\omega_3^2 - \omega_1^2)$	pitch input	$\rightarrow \tau_\theta = l \cdot U3$
$U4 = b(\omega_1^2 + \omega_3^2 - \omega_2^2 - \omega_4^2)$	yaw torque	[N·m]

Why this matters: we design controllers in terms of $U1..U4$ (thrust and three moments), which are physically intuitive, and only convert back to individual rotor speeds at the very end. This decoupling is standard practice and makes the whole problem tractable.

Verified in code: commanding all four rotors at $\omega_{\text{hover}} = 649.9 \text{ rad/s}$ gives $U1 = 5.0620 \text{ N}$ (exactly the weight) and all three moments zero — confirming the mapping and the hover point together.

4. Translational dynamics

Thrust acts along $+z_B$. To find its effect in the inertial frame we rotate it using the ZYX Euler rotation matrix

$$R = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\phi) :$$

$$R = \begin{bmatrix} c\psi c\theta & c\psi s\theta s\phi - s\psi c\phi & c\psi s\theta c\phi + s\psi s\phi \\ s\psi c\theta & s\psi s\theta s\phi + c\psi c\phi & s\psi s\theta c\phi - c\psi s\phi \\ -s\theta & c\theta s\phi & c\theta c\phi \end{bmatrix}$$

The body-frame thrust vector is $F_b = [0, 0, U]^T$. Rotating it and applying Newton's second law with gravity gives:

$$\begin{aligned}\ddot{X} &= (U/m)(\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi) \\ \ddot{Y} &= (U/m)(\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi) \\ \ddot{Z} &= -g + (U/m)(\cos \phi \cos \theta)\end{aligned}$$

Read the third equation carefully — it is the heart of Task 2. Altitude acceleration depends on thrust *multiplied by* $\cos \phi \cos \theta$. Tilt the quadcopter and you lose vertical thrust. At 30° of pitch you have lost about 13% of your lift. A naive altitude controller sags every time the vehicle manoeuvres.

5. Rotational dynamics

Euler's equations for a rigid body with a diagonal inertia tensor, plus the **gyroscopic term** from the spinning rotors:

$$\begin{aligned}\ddot{\phi} &= ((I_{yy} - I_{zz})/I_{xx}) \cdot \dot{\theta} \dot{\psi} - (I_M/I_{xx}) \cdot \dot{\theta} \dot{\Omega}_r + (1/I_{xx}) \cdot U_2 \\ \ddot{\theta} &= ((I_{zz} - I_{xx})/I_{yy}) \cdot \dot{\phi} \dot{\psi} + (I_M/I_{yy}) \cdot \dot{\phi} \dot{\Omega}_r + (1/I_{yy}) \cdot U_3 \\ \ddot{\psi} &= ((I_{xx} - I_{yy})/I_{zz}) \cdot \dot{\phi} \dot{\theta} + (1/I_{zz}) \cdot U_4\end{aligned}$$

Three distinct effects, worth naming separately:

Term	Name	Meaning
$((I_{yy} - I_{zz})/I_{xx}) \dot{\theta} \dot{\psi}$	inertial coupling	rotating about two axes creates torque about the third — this is the "coupled dynamics" the problem statement mentions
$(I_M/I_{xx}) \cdot \dot{\theta} \cdot \dot{\Omega}_r$	gyroscopic	the four spinning rotors act as a gyroscope and resist tilting
$(1/I_{xx}) \cdot U_2$	control	the moment you actually command

$\Omega_r = \omega_1 - \omega_2 + \omega_3 - \omega_4$ is the **residual rotor speed**. In perfect hover all four are equal and $\Omega_r = 0$, so the gyroscopic term vanishes. It only bites during aggressive manoeuvres.

Note that $I_{xx} = I_{yy} = 4.984 \times 10^{-3}$ but $I_{zz} = 8.958 \times 10^{-3}$ — nearly double. The quadcopter is much harder to spin about the vertical axis, which is why **yaw is always the slowest channel**.

One honest assumption you must be ready to defend

The equations above use the **Euler angle rates** ($\dot{\varphi}$, $\dot{\theta}$, $\dot{\psi}$) where strictly they should use **body angular rates** (p , q , r). The two are related by

$$\begin{bmatrix} p \\ q \\ r \end{bmatrix} = \begin{bmatrix} 1 & 0 & -\sin \theta \\ 0 & \cos \phi & \cos \theta \sin \phi \\ 0 & -\sin \phi & \cos \theta \cos \phi \end{bmatrix} \begin{bmatrix} \dot{\varphi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}$$

For small angles this matrix $\rightarrow$ identity, so $p \approx \dot{\varphi}$, $q \approx \dot{\theta}$, $r \approx \dot{\psi}$. This is the standard simplification used in the quadcopter literature, including the Mien & Tu (2024) paper the organisers cite. **It is valid near hover and for moderate angles, which is our entire operating regime.** If a judge asks about the difference, this is the answer — do not pretend the approximation is not there.

6. The state-space form

The task explicitly asks for state space. We choose **12 states** — two per degree of freedom, since each is governed by a second-order equation:

$$\mathbf{x} = [x \ \dot{x} \ y \ \dot{y} \ z \ \dot{z} \ \phi \ \dot{\phi} \ \theta \ \dot{\theta} \ \psi \ \dot{\psi}]^T$$

Input vector $\mathbf{u} = [\delta U1, U2, U3, U4]^T$.

Linearisation about hover

The equations above are nonlinear (products of states, trigonometric functions). Classical control theory needs a **linear** model, so we linearise about the hover equilibrium:

$$\text{all angles} = 0, \quad \text{all rates} = 0, \quad U1 = W = mg = 5.0620 \text{ N}$$

Applying small-angle approximations ($\sin \phi \approx \phi$, $\cos \phi \approx 1$) and dropping products of small terms:

Nonlinear	Linearised
$\ddot{x} = (U1/m)(c\phi s\theta c\psi + s\phi s\psi)$	$\ddot{x} = g \cdot \theta$
$\ddot{y} = (U1/m)(c\phi s\theta s\psi - s\phi c\psi)$	$\ddot{y} = -g \cdot \phi$
$\ddot{z} = -g + (U1/m)c\phi c\theta$	$\ddot{z} = \delta U1/m$
$\ddot{\phi} = \dots + (1/I_{xx})U2$	$\ddot{\phi} = (1/I_{xx}) \cdot U2$
$\ddot{\theta} = \dots + (1/I_{yy})U3$	$\ddot{\theta} = (1/I_{yy}) \cdot U3$
$\ddot{\psi} = \dots + (1/I_{zz})U4$	$\ddot{\psi} = (1/I_{zz}) \cdot U4$

Giving $\dot{\mathbf{x}} = \mathbf{Ax} + \mathbf{Bu}$ with the non-zero entries:

$$\begin{array}{lll}
 A(1,2) = 1 & A(2,9) = +g & (\ddot{X} = g \cdot \theta) \\
 A(3,4) = 1 & A(4,7) = -g & (\ddot{Y} = -g \cdot \phi) \\
 A(5,6) = 1 & B(6,1) = 1/m & \\
 A(7,8) = 1 & B(8,2) = 1/I_{xx} & \\
 A(9,10) = 1 & B(10,3) = 1/I_{yy} & \\
 A(11,12) = 1 & B(12,4) = 1/I_{zz} &
 \end{array}$$

$C = I_{12}$ (all states measured), $D = 0$.

The $A(2,9) = g$ and $A(4,7) = -g$ entries are the underactuation made visible. They say: the only way to accelerate horizontally is to tilt. There is no B entry that moves the vehicle sideways directly.

The system decouples into four independent channels

Channel	Equation	Transfer function	Value
Altitude	$\ddot{Z} = \delta U / m$	$1/(m s^2)$	$1/(0.516 s^2)$
Roll	$\ddot{\phi} = (1/I_{xx})U_2$	$1/(I_{xx} s^2)$	$45.1/s^2$
Pitch	$\ddot{\theta} = (1/I_{yy})U_3$	$1/(I_{yy} s^2)$	$45.1/s^2$
Yaw	$\ddot{\psi} = (1/I_{zz})U_4$	$1/(I_{zz} s^2)$	$111.6/s^2$

Every channel is a double integrator. This is the single most useful structural fact in the whole problem, and it directly drives Task 2.

7. Why "double integrator" matters so much

$G(s) = 1/(m s^2)$ has **two poles at the origin**. Consequences:

- It is marginally stable, not stable.** Apply a constant force and the output grows without bound (as t^2). Verified: a 2% thrust error makes altitude diverge quadratically.
- Proportional control alone cannot stabilise it.** With P-only, the closed loop is $m \cdot s^2 + K_p = 0$, giving poles at $\pm j\sqrt{K_p/m}$ — pure imaginary, sustained oscillation for every K_p . **Derivative action is mandatory, not optional.**
- Ziegler–Nichols is undefined here.** The ZN ultimate-gain method needs a unique gain K_u at which oscillation begins. Since it oscillates at every gain, no such K_u exists. The open-loop ZN variant also fails, since a double integrator has no S-shaped step response. *Saying this to the judges proves you understood the plant rather than applying a recipe.*

8. Verification — the evidence the model is right

Anyone can write equations. The mark is in proving they are correct. Three independent checks, all passing:

Check 1 — hover is a true equilibrium

Feed the state $\mathbf{x} = \mathbf{0}$ and input $\mathbf{U1} = \mathbf{W}$ into the nonlinear model. Every derivative must be zero.

Residual at hover $||\dot{\mathbf{x}}|| = 0.000\text{e}+00$ ✓

Check 2 — the linearisation matches the nonlinear model

Finite-difference the nonlinear dynamics about hover and compare against the hand-derived A and B matrices, entry by entry:

$\max |A - A_{\text{numerical}}| = 1.636\text{e-}12$ ✓
 $\max |B - B_{\text{numerical}}| = 3.122\text{e-}10$ ✓

This is the number to put on your slide. Agreement to twelve decimal places means the hand derivation and the code are the same system. It converts "trust our algebra" into "here is the proof".

Check 3 — controllability

$\text{rank}(\text{ctrb}(A,B)) = 12 \text{ of } 12 \rightarrow \text{fully controllable}$ ✓

All twelve states can be driven with only four inputs. Underactuated does **not** mean uncontrollable — it means you reach the horizontal states *through* the attitude states. The rank test proves the four inputs are enough.

Supporting result

Open-loop poles: all 12 at the origin

Confirms the vehicle cannot fly without feedback — which is the whole justification for Tasks 2–4.

9. The files that implement Task 1

File	What it does
<code>quad_params.m</code>	All eight Table 1 parameters, plus derived $W = 5.0620 \text{ N}$ and $\omega_{\text{hover}} = 649.9 \text{ rad/s}$, actuator limits, sample time
<code>quad_dynamics.m</code>	The full nonlinear 6-DOF model — the six equations of §4 and §5, with optional disturbance input
<code>rotor2U.m</code>	Rotor speeds $\rightarrow U1...U4$, with the $\tau = r \times F$ geometry documented
<code>task1_statespace.m</code>	Builds A, B, C, D; runs all three verification checks; forms the four channel transfer functions; saves <code>task1_model.mat</code>
<code>smoke_test.m</code>	10-point regression check of the whole package

Output: `task1_model.mat` containing `P`, `A`, `B`, `C`, `D`, `sys_full`, `sys_alt`, `G_alt`.

10. How to present Task 1 (about 3 minutes)

1. **"6 DOF, 4 actuators — it's underactuated."** Show the state vector. Say the quadcopter must tilt to translate, and point at $\ddot{\mathbf{x}} = \mathbf{g} \cdot \boldsymbol{\theta}$ in the $\mathbf{A}$ matrix.
2. **"Thrust and torques come from ω^2 ."** Show the U1–U4 mapping and mention $\tau = r \times F$.
3. **"Here are the six nonlinear equations."** Point out the $\cos \phi \cos \theta$ in the altitude equation and say it is why Task 2 needs feedback linearisation.
4. **"Linearised about hover, it decouples into four double integrators."** Show the table.
5. **"And here is the proof it is right."** Show $\max |\mathbf{A} - \mathbf{A}_{\text{num}}| = 1.6\text{e-}12$, rank 12, and the zero hover residual.
6. **"All twelve poles are at the origin, so it cannot fly open loop."** → hand over to Task 2.

Questions you will be asked

"Why 12 states?" Six degrees of freedom, each governed by a second-order differential equation, so two states each — position and velocity.

"Why linearise if you already have the nonlinear model?" Because Bode, root locus, margins and `pidtune` all assume linearity. We keep the nonlinear model for *simulation* and use the linear one for *design* — then verify the design back on the nonlinear model. Task 2's Method D does exactly that.

"Is the linear model valid for big angles?" No — it assumes small angles. That is precisely why the altitude controller uses feedback linearisation instead, which stays exact at any tilt. We verified it tracks to within 0.017 m at 0.30 rad (17°) of pitch.

"You used Euler rates instead of body rates." Correct, and deliberate. The two coincide for small angles, and it is the standard formulation used in the reference paper. It is an approximation, and here is the exact transformation matrix that relates them.

"What did you neglect?" Aerodynamic drag on the airframe (coded but set to zero to match Table 1), blade flapping, motor dynamics (rotors assumed to reach commanded speed instantly), ground effect, and airframe flexibility — the last by explicit instruction in the problem statement.